Advanced Operating Systems, Fall 2026, CSIE, NPTU

Project 1: Build a shell program.

Due Date: 2026/10/14 09:00 Turnin Code: aos.proj1

Overview

In this project, you are requested to implement a shell program (such as bash, zsh, etc.). The shell should operate in this basic way: when you type in a command (in response to its prompt), the shell creates a child process (calls fork()) that executes the command you entered (calls exec()) and then prompts for more user input when it has finished.

Program Specifications

Your shell program is basically an interactive loop: it repeatedly prints a prompt, parses the input, executes the command specified on that line of input, and waits for the command to finish. This is repeated until the user types 'exit'. Note that you must determine the name of your shell program as well as the content (and format) of the prompt (to show the working directory, user name, date, time, etc.). For convenience, I assume the name of your shell is "mysh" and the prompt is "myprompt>".

The shell can run in either interactive mode or batch mode. It enters interactive mode if invoked without arguments. The following example demonstrates interactive mode, which is invoked without arguments:

```
$ ./mysh
myprompt>
```

At this point, "mysh" is running and ready to receive commands.

When the shell is invoked with an argument, it operates in batch mode. The shell reads its input from the batch file specified by the command-line argument. The following example shows batch mode invoked by a batch file named "batch.txt":

```
$ ./mysh batch.txt
```

Note that it should not display any command prompt in batch mode.

Your shell should create a new process for each command (except for built-in commands, discussed below). It must parse the input command and run the corresponding executable. For example, if the user types 'ls -la /tmp', the shell should first search for the 'ls' program in PATH, and then run it with the given arguments '-la' and '/tmp'.

Built-in Commands

Whenever your shell receives a command, it should check if it is a built-in command. If so, it should not be executed like other programs. Instead, your shell will execute your custom handler for that built-in. For example, implementing the 'exit' built-in simply requires calling 'exit(0)' in your code, which safely exits the shell.

In this project, you should implement 'exit', 'cd', and 'path' as built-in commands:

- 'exit': When the user command is 'exit', your shell should call the 'exit()' system call with 0 as the parameter.
- 'cd': It must provide one argument (otherwise it is considered an error). To change directories, use the 'chdir()' system call with the argument specified by the user; if 'chdir()' fails, that is also considered an error.
- 'path': The 'path' command takes 0 or more arguments (separated by whitespace). A typical usage is as follows:

```
myprompt> path    /bin    /usr/bin
```

It adds '/bin' and '/usr/bin' to the shell's search path. Note that it is unable to execute any programs (except built-in commands) if the user sets path to be empty. Also note that the 'path' command always overwrites its content with the newly specified path.

I/O Redirection

Users often need to send a program's output to a file rather than the screen (stdout), or read input from a file instead of the keyboard (stdin). Your shell is requested to provide this feature through the '>' and '<' characters. Note that this concept formally referred to as I/O redirection.

Pipe

Your shell is also requested to support pipe, i.e., feed the output of a program to another program by using the ' | ' character. For example:

```
myprompt> echo shell.c | wc
```

Parallel Commands

Your shell is requested to allow users to execute multiple commands concurrently. This is implemented with the ampersand (&) operator as follows:

```
myprompt> cmd1 & cmd2 args1 args2 & cmd3 args1
```

In this case, your shell should run 'cmd1', 'cmd2', and 'cmd3' in parallel.

Program Errors

You should print the error message using the following code (whenever you encounter an error):

```
char error_message[30] = "An error has occurred\n";
write(STDERR_FILENO, error_message, strlen(error_message));
```